Saving ESM3 structures as PDB files and comparing them with 2HMG

Monomer and glycine-linked trimer: commands, numbers, viewers

4 November 2026

1 Short answers

How do I save the structure ESM3 gives me? With `write_pdb()` from `softmax/monte_carlo/decode_to_pdb.py`. It is our own helper, not an ESM function. The decode used everywhere in this repository, `decode_structure_with_mask()` in `compute_structural_metrics.py`, returns three tensors:

- `positions`, shape $(L, 37, 3)$: the coordinates of up to 37 atom types per residue (the "atom37" layout), in Å;
- `mask`, shape $(L, 37)$: which of those atoms exist;
- `plddt`, shape (L): the confidence per residue, on 0–1.

`write_pdb()` turns these into a text `.pdb` file and puts 100×pLDDT in the B-factor column, where every viewer expects it. You normally do not call it yourself: the script `decode_to_pdb.py` decodes and saves in one command (Section 3). Section 4 shows the call from your own Python code.

Is the trimer with glycine linkers in `references.py`? No. `references.py` holds only single sequences (`zero_polymer`, `zero_polymer_mature`, the controls). `decode_trimer.py` builds the trimer on the fly from `zero_polymer`:

```
unit   = ref_seq[16:529]                # residues 17..529, 513 aa
trimer = ("G" * 25).join([unit] * 3)  # 3 x 513 + 2 x 25 = 1589 residues
```

`decode_to_pdb.py` now does the same with `--copies 3 --linker 25`. Note that `decode_trimer.py` takes a *Python slice* (`--residues 16:529`, 0-based, end excluded) while `decode_to_pdb.py` takes *residue numbers* (`--residues 17:529`, 1-based, both ends included). Both mean the same 513 residues.

The trimer file is written as three chains A, B, C, one per HA copy, each numbered 17–529 like the monomer. The glycine linkers are decoded (ESM3 sees the full 1589-residue sequence) but are *not* written to the file, since they are not part of HA; their mean pLDDT is printed in the log instead.

2 What the numbers mean

RMSD (root-mean-square deviation; this is the "wmd" your supervisor meant). Pair each residue's CA atom in the prediction with the same residue's CA in the experiment, rotate

and shift the prediction onto the experiment as well as possible (the Kabsch algorithm finds the best rotation exactly), then

$$\mathrm{RMSD} = \sqrt{\tfrac{1}{N}\sum_{i=1}^{N} \|x_i^{\mathrm{pred}} - x_i^{\mathrm{exp}}\|^2}.$$

It is a typical distance in Å. About 1 Å: the same structure up to noise; 2–3 Å: same fold, some loops or a domain off; 4–8 Å: large parts placed differently; above about 10 Å for 500 residues: not the same shape. One misplaced region inflates it for the whole protein.

Core RMSD Fit, drop residues more than 2 Å off, refit, five times (what PyMOL's `align` reports by default). How well the matching part matches; always read it with the fraction of residues it kept.

Within 1/2/4/8 Å Fraction of residues within that distance after the global fit. "70% within 4 Å" is easier to read than an RMSD.

TM-score 0 to 1, independent of length. Above 0.5: same fold. Below about 0.17: like two unrelated structures.

Per-chain RMSD Each experimental chain (HA1, HA2, and for the trimer each copy) fitted on its own. Low per chain but high overall means the pieces are right but packed wrongly against each other.

pLDDT ESM3's confidence per residue: > 90 very confident, 70–90 confident, 50–70 low, < 50 roughly a guess. The script also gives Spearman(pLDDT, deviation); negative means the model is least sure exactly where it is wrong.

What to expect: HA decodes at mean pLDDT about 0.4–0.6 both as a monomer and as the G25 trimer (DEVLOG, 2026-11-04), so expect a large RMSD and a TM-score below 0.5. The comparison turns "the decoder does not fold HA" into a number against the real structure. Maltose-binding protein (Section 3.4) shows what a good match looks like.

3 Running it on astaroth

All commands are run from `softmax/monte_carlo`; adjust the first `cd` to your clone. Pull first so the scripts are there.

3.1 The experimental structure

```
cd ~/my_extendedSampling && git pull
cd softmax/monte_carlo
mkdir -p structure_compare
wget -O structure_compare/2HMG.pdb https://files.rcsb.org/download/2HMG.pdb
grep -c '^ATOM' structure_compare/2HMG.pdb   # thousands of lines = download worked
```

2HMG is the cleaved protein: separate HA1 and HA2 chains, three of each. The comparison script lists the chains it finds and which it used.

3.2 Predict and save (GPU)

Monomer (full HA0, the sequence the sampler works on, numbered 1–566):

```
nohup python -u decode_to_pdb.py --ref zero_polymer --num-steps 32 \
    --out structure_compare/ha0_pred.pdb > structure_compare/decode_ha0.log 2>&1 &
```

Trimer, exactly as `decode_trimer.py` builds it (ectodomain 17–529, three copies, 25 glycines between them):

```
nohup python -u decode_to_pdb.py --ref zero_polymer --residues 17:529 \
    --copies 3 --linker 25 --num-steps 32 \
    --out structure_compare/ha_trimer_pred.pdb \
    > structure_compare/decode_trimer.log 2>&1 &
```

Follow a log with `tail -f structure_compare/decode_trimer.log` (Ctrl-C stops watching, not the job). Each ends with `Saved -> ...`, after printing the mean pLDDT and radius of gyration (per chain and for the linkers, for the trimer). Run the two decodes one after the other if GPU memory is tight; the trimer is 1589 residues, the longest decode so far. `--num-steps` is the same setting as in the other decode scripts; HA's structure changes with it, so write the step count next to every RMSD you report.

3.3 Compare (CPU, seconds)

```
python compare_structures.py --pred structure_compare/ha0_pred.pdb \
    --exp structure_compare/2HMG.pdb --out-prefix structure_compare/ha0_vs_2HMG \
    | tee structure_compare/ha0_vs_2HMG.txt

python compare_structures.py --pred structure_compare/ha_trimer_pred.pdb \
    --exp structure_compare/2HMG.pdb --out-prefix structure_compare/trimer_vs_2HMG \
    | tee structure_compare/trimer_vs_2HMG.txt
```

How the script pairs residues:

- By sequence alignment, not by residue number, so the different numbering of our sequence and of 2HMG does not matter.
- **Monomer.** ESM3 gives one chain (HA0); 2HMG has HA1 and HA2 as separate chains. Both are aligned onto the one predicted chain. The other two copies in 2HMG would land on the same predicted residues, so they are skipped ("symmetry copy").
- **Trimer.** Residues already paired are hidden from the next alignment, so the three HA1/HA2 pairs of 2HMG land on the three predicted chains A, B, C. Which experimental copy goes with which predicted copy is arbitrary, so the script tries every order and keeps the lowest RMSD; the line "predicted protomers tried in every order" lists all of them. The trimer number therefore includes how the three copies are packed, the thing the trimer was built to test.
- To compare one predicted copy only: `--pred-chain A`. To choose experimental chains by hand: `--exp-chains A,B`.
- Residues absent from the crystal (signal peptide, transmembrane anchor, tail, disordered loops) cannot be compared and are left out of every number; the script says how many.

Files written next to the prefix:

`_superposed.pdb`	the prediction moved on top of 2HMG
`_per_residue.csv`	per residue: predicted chain and number, pLDDT, matching 2HMG chain and number, deviation
`_per_residue.png`	deviation (blue) and pLDDT (red) along the file; dashed lines separate experimental chains
`_view.cxc / .pml`	ready-made ChimeraX / PyMOL scenes

3.4 Positive control

Maltose-binding protein decodes at pLDDT 0.93; PDB 1ANF is its crystal structure.

```
wget -O structure_compare/1ANF.pdb https://files.rcsb.org/download/1ANF.pdb
nohup python -u decode_to_pdb.py --uniprot P0AEX9 --num-steps 32 \
    --out structure_compare/mbp_pred.pdb > structure_compare/decode_mbp.log 2>&1 &
python compare_structures.py --pred structure_compare/mbp_pred.pdb \
    --exp structure_compare/1ANF.pdb --out-prefix structure_compare/mbp_vs_1ANF
```

`--uniprot` needs network access to UniProt and takes the mature chain.

4 Calling it from your own Python code

If you already have the tensors from a decode inside your own script (for example a sampled sequence), save them like this. Run from `softmax/monte_carlo` so the imports resolve, as in the other scripts.

```
from compute_structural_metrics import decode_structure_with_mask
from decode_to_pdb import write_pdb
from custom_esm.utils.residue_constants import atom_types

positions, mask, plddt = decode_structure_with_mask(sampler.model, eprot, config)

write_pdb("my_structure.pdb", eprot.sequence,
          positions.float().numpy(), mask.bool().numpy(),
          plddt.float().numpy() * 100,     # pLDDT 0-100 into the B-factor column
          atom_types)
```

For a G-linked trimer, pass `segments` so each copy becomes its own chain and the linkers are left out. Each entry is (chain, first index, end index, first residue number), with 0-based indices into the decoded sequence:

```
n, L = 513, 25                          # residues per copy, glycines per linker
segments = [("A", 0,           n,           17),
            ("B", n + L,       2 * n + L,   17),
            ("C", 2 * (n + L), 3 * n + 2*L, 17)]
write_pdb("trimer.pdb", eprot.sequence,
          positions.float().numpy(), mask.bool().numpy(),
          plddt.float().numpy() * 100, atom_types, segments=segments)
```

ESM's own `ESMProtein.to_pdb()` also writes PDB files, but the decode path here never builds an `ESMProtein` with coordinates, and the helper the repository used before (`test_cleaved_complex.py --pdb`) leaves every B-factor at 1.00 and writes pLDDT to a separate `.plddt.tsv`. `write_pdb()` keeps everything in one file.

5 Looking at the structures

astaroth has no screen. Copy the folder to your laptop (from the laptop; replace the user name and path):

```
scp -r user@astaroth:~/my_extendedSampling/softmax/monte_carlo/structure_compare .
cd structure_compare
```

ChimeraX (free for academic use, `https://www.cgl.ucsf.edu/chimerax/download.html`). Quickest: *File* > *Open...* and pick `trimer_vs_2HMG_view.cxc`. 2HMG appears in grey, the prediction coloured by pLDDT (dark blue > 90, light blue 70–90, yellow 50–70, orange < 50), already on top of each other. By hand, in the command line at the bottom of the window:

```
open 2HMG.pdb                          # model #1
open ha_trimer_pred.pdb                # model #2
delete solvent
matchmaker #2/A to #1/A                # superimpose; prints an RMSD in the log
color #1 lightgray
color bfactor #2 palette alphafold
view
```

`matchmaker` fits one chain to one chain (here predicted copy A onto HA1 chain A), so its RMSD differs from the script's, which fits everything at once. It prints two values: over "pruned" pairs (like the core RMSD) and "across all pairs". `save picture.png width 2000 supersample 3` saves an image.

PyMOL (`conda install -c conda-forge pymol-open-source`). *File* > *Run Script...* and pick `trimer_vs_2HMG_view.pml`, or type:

```
load 2HMG.pdb, exp
load ha_trimer_pred.pdb, pred
remove solvent
align pred and name CA, exp and name CA, cycles=0   # all residues, no outlier
    rejection
hide everything
show cartoon
color grey70, exp
spectrum b, red_white_blue, pred, minimum=30, maximum=90
orient
```

The log line `Executive: RMSD = ...` gives the RMSD.

Browser, nothing to install. `https://molstar.org/viewer/`: drag in `2HMG.pdb` and the `_superposed.pdb` file (Mol* does not fit them for you), then set the prediction's colour theme to "Uncertainty/Disorder" to colour by the B-factor column.

6 Things that trip people up

- **Slice versus residue numbers**: `16:529` in `decode_trimer.py` is `17:529` in `decode_to_pdb.py`.
- **One chain against two**: HA0 is one chain in the prediction, HA1 + HA2 in the crystal. The script handles it; single-chain tools such as ChimeraX's `matchmaker` give different numbers.
- **Monomer against trimer**: a monomer prediction has no partner copies holding the stalk helices, so part of its disagreement with 2HMG is quaternary. Compare the per-chain RMSDs of the monomer and trimer runs.
- **Step count**: report `--num-steps` with every number.
- **Strain**: the identity line says how close 2HMG's sequence is to ours; mention it if it is well below 100%.